



PLATFORM ENGINEERS • RELEASE OPERATORS

DEVOPS MANUAL

CI, Docker, Nuitka, metadata crypto, GCP, security

Open-source Pine Script evaluation framework



CONTENTS

01 DevOps

02 Local development

03 Continuous integration

04 Release

05 Docker

06 Nuitka LSP build

07 Metadata encryption

08 GCP & Cloud Run

09 Observability

10 Security



DEVOPS

CI matrices, containers, Nuitka LSP binaries, metadata crypto, Cloud Run, and operational invariants for PYNE.

DEVOPS

Abstract

PYNE is not a single binary. It is a **multi-surface product**: a pure-Python library, a Click CLI, a pygls Language Server, a Flask Pro API, a VS Code extension, an AXIS PWA AXIS, and colocated edge workers. DevOps here means keeping those surfaces buildable, testable, and deployable without drifting from the same evaluate contract.

This tab documents the **operational graph**: local loops, GitHub Actions, release tags, Docker images, Nuitka compilation, Fernet-encrypted LSP metadata, GCP Cloud Build/Run, observability hooks, and security controls.

Conceptual model

```
flowchart TB
    subgraph Local
        DEV[make install / hatch]
        TEST[make test lint]
        RUN[make run / run-lsp]
    end
    subgraph CI["GitHub Actions"]
        LINT[lint + mypy]
        PY[test matrix 3.10-3.13]
        PWA[Bun typecheck + coverage]
        VSIX[vscode-extension package]
    end
    subgraph Artifacts
        WHEEL[pynescrypt wheel]
        LSPBIN[Nuitka pynescrypt-lsp]
        VSIXART[*vsix]
        IMG[Docker API image]
    end
    subgraph Deploy
        CR[Cloud Run Pro API]
        REL[GitHub Release + Marketplace]
    end
    DEV --> TEST --> LINT
    LINT --> PY --> PWA
    PY --> WHEEL
    REL --> LSPBIN
    REL --> VSIXART
    IMG --> CR
```

Invariant: CI green on `main` is necessary but not sufficient for a release. Tag-triggered release workflows build platform LSP binaries and VSIX; Cloud Build deploys the API image on its own pipeline.



Interface surface

Concern	Entry	Doc
Local install & loops	<code>Makefile</code> , hatch envs	Local development
PR / push CI	<code>.github/workflows/ci.yml</code>	CI
Versioned LSP + VSIX	<code>.github/workflows/release.yml</code>	Release
Pro API containers	<code>Dockerfile</code> , <code>Dockerfile.api</code> , <code>docker-compose.yml</code>	Docker
Compiled LSP	<code>scripts/build/compile.py</code> , <code>ci_build.py</code>	Nuitka build
Builtin metadata crypto	Fernet key + <code>.enc</code>	Metadata crypto
Cloud Run	<code>cloudbuild.yaml</code>	GCP
Logs / health	Flask / , gunicorn, Cloud Logging	Observability
Auth, CORS, secrets	<code>backend/middleware/auth.py</code>	Security

Internals (repo map)

Path	Role
<code>Makefile</code>	Human-facing orthography: install, test, lint, build, docker, worker
<code>pyproject.toml</code>	Hatch envs (<code>test</code> , <code>lint</code> , <code>docs</code>), optional extras (<code>lsp</code> , <code>data</code>)
<code>.github/workflows/</code>	<code>ci.yml</code> , <code>release.yml</code> , <code>axis-nightly.yml</code>
<code>scripts/build/</code>	Nuitka compile + CI build + Fernet metadata stage
<code>scripts/generate_builtin_metadata.py</code>	Regenerates LSP <code>builtin_metadata.json</code> from live builtins
<code>Dockerfile</code> / <code>Dockerfile.api</code>	Dev vs multi-stage production API images
<code>docker-compose.yml</code>	Local API (+ optional Redis / LSP profiles)
<code>cloudbuild.yaml</code>	Build → GCR push → Cloud Run deploy; optional LSP compile
<code>vscode-extension/</code>	Node 22 extension package / vsce

Invariants & edge cases

1. **Two console scripts, two entrypoints.** `pynescrypt` (Click) ≠ `pynescrypt-lsp` (pygls). Ops scripts must not conflate them.
2. **Generated artifacts are not hand-edited.** ANTLR/ASDL under `generated/` and `builtin_metadata.json` are code-derived.
3. **Fernet key is gitignored.** Without a stable `CRYPTO_KEY` / `METADATA_KEY` secret, every CI encrypt produces a different `.enc` blob (harmless functionally, bad for reproducibility).
4. **Python matrix vs local default.** CI tests 3.10–3.13; Nuitka release currently pins 3.12.
5. **AXIS nightly is not PR-blocking.** Full Playwright + security gates run on schedule (`axis-nightly.yml`).



Worked examples

```
# Fast local confidence loop
make install
make lint
make test-lsp
make build-check    # import check only, ~30s, no Nuitka compile

# API in Docker (production Dockerfile)
make docker-build
make docker-run
```

Failure modes

Symptom	Likely cause	Fix
CI lint red, local green	Different ruff/mypy versions	Match CI install pins in workflow
Metadata decrypt fails in binary	Missing key at runtime	Set <code>PYNESCRYPT_METADATA_KEY</code> or embed key at build
Cloud Run 502 after deploy	Image missing deps / gunicorn bind	Confirm <code>Dockerfile.api</code> CMD and <code>PORT=8080</code>
VSIX empty / missing	<code>npm ci</code> / vsce not run	Use <code>make build-vscode</code> or release job artifacts
Nuitka Anaconda link error	Static libpython missing	<code>conda install libpython-static</code> or keep <code>--static-libpython=no</code>

See also

- [Contributing](#)
- [LSP architecture](#)
- [Pro API contract](#)
- [pine-worker](#)



LOCAL DEVELOPMENT

Install, test, lint, and run PYNE surfaces on a developer machine — Make, hatch, Bun, and dual-language loops.

LOCAL DEVELOPMENT

Abstract

A PYNE checkout is a **polyglot workspace**: Python (src-layout package + Flask backend), TypeScript/Bun (AXIS frontend, Cloudflare worker, `pine-worker`), and Node (VS Code extension). Local DevOps is about choosing the right orthography for the surface you are changing without spinning up the entire mesh.

Conceptual model

```
flowchart LR
  SRC[src/pynescrypt] --> CLI[pynescrypt CLI]
  SRC --> LSP[pynescrypt-lsp]
  SRC --> LIB[import pynescrypt]
  BE[backend/] --> API[Flask :8080 / :5002]
  FE[frontend/] --> PWA[Bun server :8081]
  PW[pine-worker/] --> BUN[bun test]
  VSC[vscode-extension/] --> VSIX[vsce package]
```



Interface surface

Make targets (primary)

Target	Effect
<code>make install</code>	<code>pip install -e ".[lsp]"</code>
<code>make install-bun</code>	Root + <code>frontend/worker</code> Bun deps
<code>make test</code>	Frontend Bun tests + full <code>pytest tests/</code>
<code>make test-lsp</code>	<code>test_langserver.py</code> + <code>test_lsp_features.py</code>
<code>make test-backend</code>	<code>tests/test_backend.py</code>
<code>make lint</code> / <code>make fmt</code>	Ruff check / format on <code>src/</code> , <code>tests/</code> , <code>backend/</code>
<code>make typecheck</code>	Root TS + frontend worker <code>tsc</code>
<code>make build</code>	Nuitka LSP via <code>scripts/build/compile.py --jobs=4</code>
<code>make build-check</code>	Fast import resolution only (~30s)
<code>make build-vscode</code>	Compile + package VSIX
<code>make run</code>	<code>python -m backend.app</code>
<code>make run-lsp</code>	<code>python -m pynescrypt.langserver</code>
<code>make run-frontend</code>	AXIS PWA on <code>:8081</code> (expects API on <code>:5002</code>)
<code>make docker-build</code> / <code>docker-run</code>	Production API image / compose

Hatch equivalents

```
hatch run test:test
hatch run test:test-cov
hatch run lint:style
hatch run lint:typing
hatch run lint:gen-parser # ANTLR regeneration – see grammar guide
```

See also `CONTRIBUTING.md` and `AGENTS.md` for hatch-first workflows.

Internals

Path	Why it matters locally
<code>src/pynescrypt/</code>	Editable package root (src-layout)
<code>tests/conftest.py</code>	Parametrizes <code>pynescrypt_filepath</code> over every <code>tests/data/builtin_scripts/*.pine</code> (~hundreds of cases)
<code>tests/data/library/</code>	Reference corpus only — not auto-parametrized
<code>backend/requirements.txt</code>	Extra deps for Pro API (Flask, numpy, matplotlib, ...)
<code>pine-worker/</code>	Colocated TS port; Bun tests independent of Python wheel



Python requirements

- **Requires-Python:** `>=3.10` (`pyproject.toml`)
- **CI matrix:** 3.10–3.13
- **Recommended local:** 3.12 or 3.13 with a `virtualenv` / `hatch env`

Optional extras

```
pip install -e ".[lsp]"          # pygls + lsprotocol
pip install -e ".[dev-lsp]"      # + pytest-lsp for e2e LSP
pip install -e ".[data]"         # ccxt for data providers
pip install -r backend/requirements.txt
```

Invariants & edge cases

1. `from __future__ import annotations` is mandatory on every new Python file (ruff isort `required-imports`).
2. **Do not edit generated grammar.** Change only `src/pynescrypt/ast/grammar/antlr4/resource/*.g4` , then regenerate.
3. **No stale backups in `src/`** . Live `builder.py` and `technical.py` are sole sources of truth.
4. **Corpus fixture cost.** Any test using `pinescript_filepath` multiplies across the builtin script corpus. Narrow with `--example-scripts-dir=...` when iterating.
5. **Console scripts are separate.** Installing `.[lsp]` gives both `pynescrypt` and `pynescrypt-lsp` ; invoking the wrong one is a common first-day footgun.

Worked examples

Library + parse round-trip

```
make install
python -c "from pynescrypt.ast.helper import parse, unparse; print(unparse(parse('//@version=6\nninc
```

Pro API + AXIS

```
# terminal 1
make run
# terminal 2
make run-frontend
# open http://127.0.0.1:8081 - /run hits backend on :5002 per compose/docs convention
```

LSP without Nuitka

```
make install
make run-lsp
# point editor to stdio server: python -m pynescrypt.langserver
```




pine-worker only

```
cd pine-worker
bun install
bun test
bun run typecheck
```

Failure modes

Symptom	Cause	Mitigation
ModuleNotFoundError: pynescrypt	Not editable-installed	make install
Hundreds of unexpected test failures	Editing shared conftest fixture	Isolate unit tests; avoid corpus fixture
Frontend fails calling API	Backend not on expected port	Start make run ; check CORS / ALLOWED_ORIGINS
antlr4 not found	CLI not on PATH	Use hatch lint:gen-parser or full path to antlr4
Ruff I import errors	Missing future annotations	Add from __future__ import annotations

See also

- [CI](#)
- [Docker](#)
- [Nuitka build](#)
- [Contributing](#)
- [Installation \(end user\)](#)



CONTINUOUS INTEGRATION

GitHub Actions matrix for lint, Python tests, backend, VSIX, AXIS PWA, e2e smoke, and nightly gates.

CONTINUOUS INTEGRATION

Abstract

CI is the **public invariant check** for the monorepo-like layout. A single PR can touch the Python evaluator, the Flask API, the VS Code extension, the AXIS PWA, and a Cloudflare Worker typecheck — the workflow graph is deliberately fan-out with concurrency cancellation on newer pushes to the same ref.

Primary definition: `.github/workflows/ci.yml` . Nightly AXIS expansion lives in `axis-nightly.yml` . Release builds are separate (`release.yml`).

Conceptual model

```
flowchart TB
    PUSH[push / PR → main] --> CONC[concurrency: ci-ref]
    CONC --> LINT[lint]
    CONC --> TEST[test matrix 3.10-3.13]
    CONC --> BE[backend-test]
    CONC --> VSC[vscode-ext-test]
    CONC --> PWA[pwa Bun]
    CONC --> E2E[axis-e2e smoke]
    CONC --> CF[cf-worker typecheck]
    NIGHT[schedule 04:00 UTC] --> FULL[axis-nightly full e2e + security]
```

Interface surface

Triggers

Workflow	On	Purpose
<code>ci.yml</code>	push/PR to <code>main</code>	Required-path confidence
<code>axis-nightly.yml</code>	cron <code>0 4 * * *</code> + <code>workflow_dispatch</code>	Full Playwright + security + coverage gate
<code>release.yml</code>	tags <code>v*</code> + dispatch	LSP binaries + VSIX + GitHub Release



ci.yml jobs

Job	Runtime	What it asserts
lint	Python 3.13	<code>ruff check src/ tests/ ; mypy src/ --ignore-missing-imports</code>
test	Matrix 3.10–3.13	Editable <code>.[lsp]</code> ; LSP unit/e2e tests; backend tests; Codecov upload on 3.13 only
backend-test	3.13	Explicit Flask/numpy/matplotlib install + <code>test_backend.py</code>
vscode-ext-test	Node 22	<code>npm ci</code> → compile → <code>vsce package</code> → upload VSIX artifact (7-day)
pwa	Bun 1.3	Root + worker install, dual <code>tsc</code> , frontend coverage gate + security tests; upload lcov
axis-e2e	Bun 1.3	Playwright Chromium smoke (<code>test:e2e:smoke</code>)
cf-worker	Bun 1.3	<code>frontend/worker</code> typecheck only

Concurrency:

```
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
```

Nightly (`axis-nightly.yml`)

- Full e2e suite (`bun run test:e2e`) with report artifact on failure
- Coverage gate + security tests (stricter than PR smoke)

Internals

Path	Role
<code>.github/workflows/ci.yml</code>	PR/main fan-out
<code>.github/workflows/axis-nightly.yml</code>	Scheduled AXIS depth
<code>.github/workflows/release.yml</code>	Tag release (see Release)
<code>frontend/scripts/check-coverage.mjs</code>	Coverage gate helper for AXIS
<code>codecov</code> action	Optional; <code>fail_ci_if_error: false</code>

Secrets referenced in CI surface

Secret	Used by	Notes
<code>VSCE_PAT</code>	VSIX package / publish	Extension packaging
<code>METADATA_KEY</code>	Release / Cloud Build encrypt	Stable Fernet key for metadata
<code>GITHUB_TOKEN</code>	Release upload	Default Actions token

` — CI `test` job currently does not run the full `tests/` corpus on every matrix cell; it prioritizes LSP + backend. Local `make test` is broader.



Invariants & edge cases

1. **Fail-fast is off** on the Python matrix — one version failure does not hide others.
2. **Codecov only on 3.13** to avoid noisy multi-upload; coverage XML path depends on pytest-cov configuration in the job.
3. **PWA install is resilient:** `bun install --frozen-lockfile || bun install` — lockfile drift should still be fixed, not papered forever.
4. **VSIX artifacts are not the Marketplace.** Packaging on every PR proves buildability; publish is release-gated.
5. **Worker typecheck ≠ deploy.** `cf-worker` job does not call `wrangler deploy`.

Worked examples

Reproduce lint locally as CI does

```
pip install ruff flake8 mypy types-python-dateutil antlr4-python3-runtime
ruff check src/ tests/ --output-format=github
mypy src/ --ignore-missing-imports
```

Reproduce Python job slice

```
pip install -e ".[lsp]"
pip install pytest pytest-cov pytest-xdist pytest-asyncio
python -m pytest tests/test_langserver.py tests/test_lsp_features.py -v --tb=short
python -m pytest tests/test_backend.py -v --tb=short
```

Reproduce PWA job

```
bun install --frozen-lockfile
cd frontend/worker && bun install --frozen-lockfile && bunx tsc --noEmit
cd ../.. && bunx tsc -p tsconfig.json
cd frontend && bun install && bun run test:coverage:gate && bun run test:security
```

Failure modes

Symptom	Likely cause	Fix
Matrix red only on 3.10	Version-specific typing / API	Guard with <code>sys.version_info</code> or drop unsupported API
Playwright smoke flake	Timing / missing deps	Install chromium with deps; re-run; check nightly for full suite
VSIX job fails on <code>vsce</code>	Node/engine mismatch	Pin Node 22 as workflow does
Coverage gate red	New untested AXIS paths	Add unit tests or adjust gate intentionally
Cancelled mid-run	Newer push to same branch	Expected concurrency behavior

See also

- [Release](#)
- [Local development](#)



- Observability
- Security



RELEASE

Tag-driven LSP multi-platform Nuitka binaries, VSIX packaging, GitHub Releases, and Marketplace publish.

RELEASE

Abstract

A PYNE release is a **multi-artifact event**, not a single wheel upload. The primary automation (`.github/workflows/release.yml`) builds platform-native Language Server binaries with Nuitka, packages the VS Code extension, attaches them to a GitHub Release, and optionally publishes to the VS Code Marketplace.

Library wheels may still be built separately via hatch / PyPI workflows; the documented release pipeline optimizes for **editor distribution** (LSP + VSIX).

Conceptual model

```
sequenceDiagram
    participant Dev as Maintainer
    participant GH as GitHub
    participant Build as build-lsp matrix
    participant VSC as build-vscode
    participant Rel as release job
    participant Mkt as Marketplace
    Dev->>GH: git tag vX.Y.Z && push
    GH->>Build: linux / windows / macos
    GH->>VSC: Node 22 vsce package
    Build->>Rel: artifacts
    VSC->>Rel: VSIX
    Rel->>GH: softprops/action-gh-release
    VSC->>Mkt: vsce publish (on tag)
```

Interface surface

Triggers

Event	Behavior
Push tag matching <code>v*</code>	Full build + create release + marketplace publish
<code>workflow_dispatch</code> with <code>version</code> input	Manual build path (release job still gated on tag push condition)



Jobs

Job	Matrix / OS	Output
build-lsp	ubuntu-latest, windows-latest, macos-latest	Named artifacts: <code>pynescrypt-lsp-linux-x86_64</code> , <code>...-windows-x86_64.exe</code> , <code>...-macos-arm64</code>
build-vscode	ubuntu-latest	<code>pynescrypt-vscode-extension</code> VSIX
release	needs both; only if tag push	GitHub Release body + asset globs
publish-vscode	needs VSIX; only if tag push	<code>vsce publish --pat</code>

Environment

```
env:  
  PYTHON_VERSION: "3.12"  
  NUITKA_JOBS: 4
```

Build steps rely on:

```
pip install nuitka==2.0.* cryptography  
python scripts/build/compile.py --no-encrypt --check # metadata stage  
python scripts/build/ci_build.py --jobs 4  
# CRYPTO_KEY from secrets.METADATA_KEY
```

Local make-side packaging

```
make build # scripts/build/compile.py --jobs=4  
make build-vscode # npm install && compile && vsce package
```

Artifacts land under `dist/lsp/`, `dist/vsix/`, and `vscode-extension/*.vsix` depending on script path.

Internals

Path	Role
<code>.github/workflows/release.yml</code>	Orchestration
<code>scripts/build/ci_build.py</code>	CI-oriented Nuitka + metadata encrypt
<code>scripts/build/compile.py</code>	Local/full compile options (<code>--onefile</code> , <code>--standalone</code> , <code>--check</code>)
<code>vscode-extension/package.json</code>	Extension version / engines
<code>src/pynescrypt/__about__.py</code>	Hatch dynamic version for Python package



Release asset contract (from release body)

Artifact	Install sketch
<code>pynescrypt-lsp-linux-x86_64</code>	<code>chmod +x → /usr/local/bin/pynescrypt-lsp</code>
<code>pynescrypt-lsp-windows-x86_64.exe</code>	Place on PATH
<code>pynescrypt-lsp-macos-arm64</code>	Place on PATH (Apple Silicon runners)
<code>pynescrypt-*.vsix</code>	<code>code --install-extension pynescrypt-*.vsix</code>

Invariants & edge cases

1. **CRYPTO_KEY** must be stable across builds if you care about byte-identical `builtin_metadata.json.enc`. Supply `secrets.METADATA_KEY`.
2. **Tag name is the version source** for the GitHub Release title (`v` prefix stripped).
3. **Marketplace publish needs VSCE_PAT**. Without it, packaging may still succeed while publish fails.
4. **Artifact retention is short** (5 days on build jobs) — releases must attach files immediately.
5. **macOS artifact is ARM64** from `macos-latest` — Intel Mac users may need separate builds if required.
6. **Draft vs prerelease**: workflow currently sets `draft: false`, `prerelease: false` for tag releases.

Worked examples

Cut a release

```
# ensure main is green
git checkout main && git pull
# bump versions in extension / __about__ as needed
git tag -a v0.3.0 -m "v0.3.0"
git push origin v0.3.0
# watch Actions → Build & Release LSP
```

Smoke-test a downloaded binary

```
chmod +x pynescrypt-lsp-linux-x86_64
./pynescrypt-lsp-linux-x86_64 --help
# or wire stdio into an editor client
```




Failure modes

Symptom	Cause	Fix
Binary not found after Nuitka	Path layout drift in <code>ci_build.py</code>	Inspect <code>dist/</code> ; fix finder step
Decrypt errors in field	Key mismatch between encrypt and runtime	Align <code>METADATA_KEY</code> with embedded key strategy
Empty GitHub Release assets	Artifact download path mismatch	Match <code>files:</code> globs to download-artifact layout
Marketplace 401	Bad/expired <code>VSCE_PAT</code>	Rotate PAT with Marketplace scopes
Windows <code>.exe</code> not marked executable	Finder uses <code>-type f -executable</code>	Fallback non-executable find branch in workflow

See also

- [Nuitka build](#)
- [Metadata crypto](#)
- [CI](#)
- [VS Code extension](#)



DOCKER

Dev and production container images for the PYNE Pro API, compose profiles, and Cloud Run image contract.

DOCKER

Abstract

Containers package the **Pro API** (Flask + evaluator) for reproducible local runs and Cloud Run deploys. Two Dockerfiles express different trade-offs: a simple single-stage image for development, and a multi-stage production image with non-root user, gunicorn, and healthchecks.

Compose wires volume mounts and optional Redis/LSP profiles for desk development.

Conceptual model

```
flowchart LR
    subgraph Images
        D1[Dockerfile single-stage]
        D2[Dockerfile.api multi-stage]
    end
    subgraph Runtime
        DEV[python -m backend.app]
        PROD[gunicorn backend.app:app]
    end
    D1 --> DEV
    D2 --> PROD
    PROD --> CR[Cloud Run :8080]
    COMPOSE[docker-compose.yml] --> D1
    MAKE[make docker-build] --> D2
```

Interface surface

Dockerfiles

File	Audience	Process model
Dockerfile	Local / compose	CMD ["python", "-m", "backend.app"], Python 3.12-slim, exposes 8080
Dockerfile.api	Production / make docker-build	Multi-stage; USER appuser ; gunicorn 2 workers × 4 threads; healthcheck curl

Make / compose

```
make docker-build    # docker build -f Dockerfile.api -t pynescrypt-api .
make docker-run      # docker compose up api --build
```

docker-compose.yml services:



Service	Profile	Notes
api	default	Port 5002:8080 , FLASK_ENV=development , ro mounts of src/ + backend/
redis	with-redis	redis:7-alpine on 6379
lsp	with-lsp	Same image, python -m pynescrypt.langserver

Network name: pynescrypt-dev .

Cloud Build image tags

From cloudbuild.yaml :

```
gcr.io/$PROJECT_ID/pynescrypt/pynescrypt-pro-api:$COMMIT_SHA
gcr.io/$PROJECT_ID/pynescrypt/pynescrypt-pro-api:latest
```

Internals

Production stages (Dockerfile.api)

1. **Builder:** install build-essential , pip install --prefix=/install -r backend/requirements.txt
2. **Runtime:** copy install prefix, copy backend/ + src/ , pip install . , drop privileges to appuser
3. **Env:** PORT=8080 , FLASK_ENV=production
4. **Healthcheck:** HTTP GET / every 30s
5. **CMD:**

```
gunicorn --bind :8080 --workers 2 --threads 4 --timeout 60 backend.app:app
```

Dev image (Dockerfile)

Copies backend/requirements.txt , pyproject.toml , README/LICENSE, installs requirements, copies src + backend , pip install . , exposes 8080.

Compose healthchecks

API: curl -f http://localhost:8080/ (inside container). Redis: redis-cli ping .

Invariants & edge cases

1. **Host port 5002 vs container 8080.** AXIS local docs assume API on 5002 when using compose; Cloud Run uses 8080.
2. **Read-only mounts in compose** mean image reinstall is required for dependency changes; source edits hot-reload only if the process reloads Python modules.
3. **Production image installs the package non-editable** — good for immutability, bad for live code mount (use compose + Dockerfile for that).
4. **Non-root appuser** cannot write arbitrary paths; configure API_KEY_STORE to a writable volume if using file-backed keys.
5. **Cloud Build currently builds context . with default Dockerfile** in cloudbuild.yaml — confirm which Dockerfile is intended for production deploys when changing images. `` if Cloud Build should pin -f



```
Dockerfile.api .
```

Worked examples

Production-like local API

```
docker build -f Dockerfile.api -t pynescrypt-api .
docker run --rm -p 8080:8080 \
  -e FLASK_ENV=production \
  -e ALLOWED_ORIGINS=http://localhost:8081 \
  pynescrypt-api
curl -s http://127.0.0.1:8080/ | jq .
```

Compose with Redis

```
docker compose --profile with-redis up --build
```

Optional LSP container

```
docker compose --profile with-lsp up lsp
```

Failure modes

Symptom	Cause	Fix
Healthcheck fails	curl missing in image	Production Dockerfile installs curl; slim custom images may not
Permission denied writing keys	Running as <code>appuser</code>	Mount writable volume; set <code>API_KEY_STORE</code>
Import errors after mount	PYTHONPATH / non-editable install	Prefer compose dev Dockerfile + <code>pip install -e</code> patterns
OOM under load	Single small instance	Raise Cloud Run memory; reduce concurrency
CORS failures from PWA	Origin not allowed	Set <code>ALLOWED_ORIGINS</code>

See also

- [GCP](#)
- [Security](#)
- [Pro API lifecycle](#)
- [Local development](#)

NUITKA LSP BUILD

Compile pynescrypt-lsp with Nuitka — onefile vs standalone, CI build script, Anaconda quirks, and artifact layout.

NUITKA LSP BUILD

Abstract

The Language Server can run as pure Python (`python -m pynescrypt.langserver`) or as a **Nuitka-compiled binary** for editor distribution. Compilation freezes the pygls server, providers, and (when configured) encrypted builtin metadata into a deployable artifact under `dist/`.

Primary tools: `scripts/build/compile.py` (local) and `scripts/build/ci_build.py` (CI/Cloud Build).

Conceptual model

```

flowchart TB
    META[generate_builtin_metadata.py] --> JSON[builtin_metadata.json]
    JSON --> ENC[Fernet encrypt → .enc + .sha256]
    JSON --> PLAIN[plaintext for dev]
    SRC[src/pynescrypt/langserver] --> NUITKA[nuitka --onefile|--standalone]
    ENC --> NUITKA
    NUITKA --> BIN[dist/.../pynescrypt-lsp]
    BIN --> VSIX[optional VSIX bundle]
```

Interface surface

Commands

```

# Prerequisites
pip install nuitka cryptography

# Full onefile build (default path via Make)
make build
# ≡ python scripts/build/compile.py --jobs=4

# Fast import check only (~30s)
make build-check
# ≡ python scripts/build/compile.py --check

# Explicit modes
python scripts/build/compile.py --standalone
python scripts/build/compile.py --onefile --jobs 8
python scripts/build/ci_build.py --jobs=4
```



Build modes (approx. wall time)

Mode	Time	Notes
<code>--check</code>	~30s	Import resolution / no full compile
<code>--standalone</code>	5–15 min	Directory distribution, faster iteration
<code>--onefile</code>	10–30 min	Self-extracting single binary
CI 8 cores	5–15 min	High parallelism

Expected layout

```
dist/
├─ lsp/
│   └─ pynescrypt-lsp          # onefile (path may vary by script)
├─ vsix/
│   └─ pynescrypt-lsp.vsix     # optional bundle
└─ pynescrypt-lsp             # ci_build final move target (script-dependent)
```

Also see `scripts/build/README.md`.

Internals

Key Nuitka flags (`compile.py`)

- `--static-libpython=no` — default; critical for many Linux/Anaconda envs
- `--python-flag=no_site,no_docstrings`
- `--include-data-dir=.../providers=pynescrypt/langserver/providers`
- `--lto=auto` , `--jobs=N`
- `--onefile` or `--standalone`
- `--follow-imports` on local compile path

Entry module

Compilation targets `src/pynescrypt/langserver` (package entry), product name `pynescrypt-lsp`.

Metadata stage

Before compile, scripts may:

1. Run `scripts/generate_builtin_metadata.py` if plaintext metadata missing
2. Fernet-encrypt to `builtin_metadata.json.enc`
3. Write truncated SHA256 sidecar
4. Write `scripts/build/.metadata.key` (gitignored) or consume `CRYPTO_KEY` env in CI

Anaconda

If static libpython is unavailable:

```
conda install libpython-static
# or rely on --static-libpython=no (already the default in compile.py)
```



Invariants & edge cases

1. **C compiler required** (gcc/clang/MSVC). Containers need `build-essential` or equivalent if compiling inside Docker.
2. **Python 3.10+**; release workflow pins **3.12**.
3. **Providers data dir must ship** — completion/hover collapse without metadata.
4. **Onfile startup cost** includes extract-to-temp; standalone is snappier for local soak tests.
5. **Do not commit** `scripts/build/.metadata.key`.

Worked examples

Local onefile with explicit jobs

```
python scripts/build/compile.py --jobs "$(nproc)"
find dist -name 'pynescrypt-lsp*' -type f
```

CI-shaped build

```
python scripts/build/ci_build.py --jobs=4
```

Verify without compile

```
python scripts/build/compile.py --check
```

Failure modes

Symptom	Cause	Fix
Link error on libpython	Anaconda static lib missing	Install static lib or keep <code>--static-libpython=no</code>
Binary missing builtins	Data dir not included	Confirm <code>--include-data-dir</code> providers path
Huge binary	Full stdlib pull-in	Review Nuitka plugins / unused deps
CI timeout	Low cores / onefile	Raise jobs; use larger runner; prefer standalone intermediate
Decrypt fail at runtime	Key not embedded / env unset	See Metadata crypto

See also

- [Metadata crypto](#)
- [Release](#)
- [LSP builtin metadata](#)



METADATA ENCRYPTION

Fernet-encrypted LSP builtin_metadata.json — key lifecycle, CI secrets, integrity hashes, and runtime decrypt.

METADATA ENCRYPTION

Abstract

LSP completion, hover, and signature help depend on a large **builtin metadata** document derived from the live evaluator surface. In open development the JSON is plaintext and git-tracked. For **compiled Nuitka binaries**, the build pipeline can ship an encrypted blob so the binary does not leave a trivially scrapable metadata file on disk.

Mechanism: **Fernet** (symmetric, `cryptography` package) + optional SHA256 integrity sidecar.

Conceptual model

```
flowchart LR
    GEN[generate_builtin_metadata.py] --> PLAIN[providers/builtin_metadata.json]
    PLAIN --> FERNET[Fernet.encrypt]
    KEY[.metadata.key or CRYPTO_KEY] --> FERNET
    FERNET --> ENC[builtin_metadata.json.enc]
    PLAIN --> SHA[builtin_metadata.json.sha256]
    ENC --> LOAD_ENC[metadata_decrypt.load_encrypted_metadata]
    KEY --> LOAD
    SHA --> LOAD
    PLAIN --> DEV[get_metadata_cached prefers plaintext]
```

Interface surface

Files

Path	Tracked?	Role
src/pynescrypt/langserver/providers/builtin_metadata.json	yes	Plaintext for dev / hatch
.../builtin_metadata.json.enc	often yes	Encrypted payload for binaries
.../builtin_metadata.json.sha256	yes	Truncated SHA256 of plaintext
scripts/build/.metadata.key	no (gitignored)	Local Fernet key material
scripts/generate_builtin_metadata.py	yes	Regenerates JSON from code



Environment variables

Variable	Context	Meaning
<code>CRYPTO_KEY</code>	Build scripts / CI / Cloud Build	Key bytes for encrypt stage
<code>PYNESCRIPT_METADATA_KEY</code>	Runtime decrypt fallback	Env-supplied key if file missing
GitHub <code>secrets.METADATA_KEY</code>	Actions	Stable secret → <code>CRYPTO_KEY</code>
Cloud Build <code>_METADATA_KEY</code>	<code>cloudbuild.yaml</code> substitution	<code>CRYPTO_KEY=\${_METADATA_KEY}</code>

Code endpoints

- Encrypt stage: `scripts/build/compile.py` (`encrypt_metadata`), `scripts/build/ci_build.py` (`stage_metadata`)
- Decrypt: `src/pynescrypt/langserver/providers/metadata_decrypt.py`
- Loader preference: plaintext if present, else encrypted (`get_metadata_cached`)

Internals

Generation (always code-derived)

```
python scripts/generate_builtin_metadata.py
```

Do **not** hand-edit the JSON for permanent feature work — re-run the generator after adding builtins.

Encrypt (local sketch)

```
from cryptography.fernet import Fernet
key = Fernet.generate_key() # or load stable secret
fernet = Fernet(key)
encrypted = fernet.encrypt(plaintext_bytes)
```

Build scripts chmod the key file to `0o600` when writing `.metadata.key`.

Decrypt integrity check

After Fernet decrypt, if a `.sha256` sidecar exists, the loader compares `sha256(plaintext)[:16]` and raises on mismatch.

Dev vs binary

Mode	Behavior
Editable install / repo checkout	Prefer plaintext JSON
Nuitka onefile	Encrypted data dir + key resolution via MEIPASS / env
Missing both	<code>FileNotFoundError</code> directing to compile script



Invariants & edge cases

1. **Stable key $\Rightarrow$ reproducible** `.enc` . Random key every CI run yields a different ciphertext even if plaintext is identical — not a functional bug, but pollutes diffs and caches.
2. **Plaintext remains in git** for open-source transparency of the *language surface*. Encryption protects the *bundled binary layout*, not the secret of the API catalog.
3. **Truncated hash is integrity, not secrecy**. It detects bitrot / wrong plaintext, not attackers with the key.
4. **Never commit** `.metadata.key` . Rotate if leaked; regenerate `.enc` with the new key for binary builds.
5. **After adding builtins**: regenerate metadata, re-encrypt with the **same** CI key, commit updated JSON (and `.enc` if tracked).

Worked examples

Regenerate + local encrypt

```
python scripts/generate_builtin_metadata.py
pip install cryptography
python scripts/build/compile.py --check # stages metadata paths as configured
```

CI snippet

```
- name: Build LSP binary
  run: python scripts/build/ci_build.py --jobs 4
  env:
    CRYPTO_KEY: ${ secrets.METADATA_KEY }
```

Runtime with env key

```
pynescrypt-lsp
```

Failure modes

Symptom	Cause	Fix
No metadata decryption key found	Missing file + env	Supply <code>PYNESCRIPT_METADATA_KEY</code> or rebuild with key
Metadata integrity check failed	Stale hash or wrong ciphertext	Re-encrypt from current plaintext; commit both
Completion empty in binary only	Data dir not included in Nuitka	Fix <code>--include-data-dir</code>
Divergent <code>.enc</code> every CI	Unset <code>CRYPTO_KEY</code>	Configure <code>METADATA_KEY</code> secret

See also

- [Nuitka build](#)



- LSP builtin metadata
- Release
- Security



GCP & CLOUD RUN

Cloud Build pipeline, Cloud Run Pro API sizing, cost model, and substitution secrets for PYNE deployments.

GCP & CLOUD RUN

Abstract

Production-shaped hosting for the Pro API targets **Google Cloud Run**, fed by **Cloud Build** (`cloudbuild.yaml`). The pipeline builds a container, pushes multi-tags to Container Registry, deploys a managed service, and can optionally compile the LSP binary in the same build graph.

This page codifies the as-checked-in config and the cost envelope from `docs/gcp_cost_estimate.md` — treat dollar figures as planning estimates, not invoices.

Conceptual model

```
flowchart LR
  GIT[push main] --> CB[Cloud Build]
  CB --> DOCKER[docker build]
  DOCKER --> GCR[gcr.io project/repo/service]
  GCR --> RUN[Cloud Run deploy]
  CB --> LSP[optional Nuitka ci_build]
  RUN --> SQL[(Cloud SQL optional)]
  RUN --> REDIS[(Memorystore optional)]
  RUN --> GCS[Cloud Storage charts]
```

Interface surface

Substitutions (`cloudbuild.yaml`)

Substitution	Default	Meaning
<code>_SERVICE_NAME</code>	<code>pynescrypt-pro-api</code>	Cloud Run service
<code>_REGION</code>	<code>us-central1</code>	Deploy region
<code>_REPOSITORY</code>	<code>pynescrypt</code>	Image path segment
<code>_METADATA_KEY</code>	(secret/subst)	Fernet key → <code>CRYPTO_KEY</code> for LSP stage

Build steps

- build** — `docker build` with BuildKit; tags `$COMMIT_SHA` + `latest` ; cache-from `latest`
- push** — `docker push --all-tags`
- deploy** — `gcloud run deploy` with managed platform flags
- build-lsp** — `python:3.12-slim` image runs `scripts/build/ci_build.py --jobs=4` with `CRYPTO_KEY=${_METADATA_KEY}`



Cloud Run flags (as configured)

Flag	Value	Rationale
<code>--allow-unauthenticated</code>	on	Public HTTP API (auth via API keys at app layer)
<code>--min-instances</code>	0	Scale to zero
<code>--max-instances</code>	10	Cap spend / blast radius
<code>--memory</code>	512Mi	Evaluator headroom
<code>--cpu</code>	1	Single vCPU per instance
<code>--concurrency</code>	10	Parallel requests per instance
<code>--timeout</code>	60s	Aligns with gunicorn timeout
<code>--set-env-vars</code>	<code>FLASK_ENV=production</code>	App mode

Machine / budget knobs

```
options:
  machineType: E2_HIGHCPU_8
  logging: CLOUD_LOGGING_ONLY
  timeout: 1200s
```

Internals

Path	Role
<code>cloudbuild.yaml</code>	Pipeline definition
<code>Dockerfile</code> / <code>Dockerfile.api</code>	Image recipes (confirm which Cloud Build uses)
<code>backend/app.py</code>	Health / , CORS, body size limit
<code>docs/gcp_cost_estimate.md</code>	Scenario costs (April 2026 draft)

Cost envelope (planning)

Scale	Rough monthly	Notes
Free / hobby	~\$0	Within Cloud Run free tier if tiny
MVP (~10 pro users)	~\$50	Cloud Run + micro SQL
Growth (~50)	~\$120	Add Redis caching
Scale (~200)	~\$350	Dominated by evaluator CPU-sec

Dominant cost driver: **script execution CPU time** (~2–5 CPU-sec per typical backtest over 1k bars). Memory ~200–400MB per concurrent Python worker.

Free-tier leverage (GCP)

- Cloud Run: 2M requests, 400K CPU-sec, 200K GB-sec / month
- Cloud Build: 120 build-minutes / day



- [Artifact Registry](#) / GCS small free slices

Invariants & edge cases

1. **App-layer auth still required** when Cloud Run allows unauthenticated invoke — see [Security](#).
2. **Scale-to-zero cold starts** add latency; min-instances=1 trades money for snappiness.
3. **LSP build step does not deploy the binary** to Cloud Run; it is a release-adjacent compile in the same YAML.
4. **512Mi may be tight** for large OHLCV payloads — watch OOM kills; raise memory before raising concurrency.
5. **Cost estimate doc** also lists Railway / Render / Fly / self-host alternatives if GCP is overkill.

Worked examples

Manual deploy sketch (operator)

```
gcloud builds submit --config cloudbuild.yaml \  
  --substitutions=_METADATA_KEY="$METADATA_KEY"
```

Smoke health after deploy

```
curl -s "https://SERVICE-URL/" \  
# expect JSON status healthy, service pynescrypt-pro-api
```

Failure modes

Symptom	Cause	Fix
Deploy timeout	Nuitka step + docker on 20 min budget	Split LSP compile to release pipeline
429 / queue	max-instances or concurrency	Raise caps carefully; add queue
High bill	No cache; chatty clients	Redis TTL; Cloud Tasks for backtests
Auth bypass illusion	Unauthenticated Cloud Run	Enforce API keys in Flask
Image pull errors	Wrong project / registry perms	IAM on GCR + Cloud Run service agent

See also

- [Docker](#)
- [Observability](#)
- [Security](#)
- [Pro API](#)



OBSERVABILITY

Health endpoints, gunicorn process model, Cloud Logging, coverage artifacts, and operational signals for PYNE.

OBSERVABILITY

Abstract

PYNE's production surface today optimizes for **simple, hostable signals** rather than a full OpenTelemetry mesh: HTTP health checks, process managers (gunicorn), cloud platform logs, CI artifacts (coverage, Playwright reports), and application-level API key usage counters. Treat this page as the map of *what exists in-repo*, not a claim of enterprise APM parity.

Conceptual model

```
flowchart TB
    CLIENT[Client] --> LB[Cloud Run / local]
    LB --> GUNI[gunicorn workers]
    GUNI --> APP[Flask backend.app]
    APP --> HEALTH["GET /"]
    APP --> RUN["POST /run ..."]
    APP --> KEYS[APIKeyStore counters]
    GUNI --> LOGS[stdout / Cloud Logging]
    CI[GitHub Actions] --> ART[coverage lcov / playwright report]
```

Interface surface

Health

backend/app.py exposes:

```
GET /
→ { "status": "healthy", "service": "pynescrypt-pro-api", ... }
```

Used by:

- Dockerfile.api HEALTHCHECK (curl -f http://localhost:8080/)
- docker-compose.yml api healthcheck
- Load balancers / Cloud Run startup-ish probes (platform-dependent)

Process model (production)

```
gunicorn --bind :8080 --workers 2 --threads 4 --timeout 60 backend.app:app
```



Knob	Effect on signals
workers	Process isolation; multiplies memory
threads	Concurrent requests within worker
timeout 60s	Aligns with Cloud Run <code>--timeout=60s</code> — long evals die predictably

Logs

Environment	Where logs go
Local <code>make run</code>	Process stdout/stderr
Docker	Container logs (<code>docker logs</code> , <code>compose</code>)
Cloud Build	<code>logging: CLOUD_LOGGING_ONLY</code>
Cloud Run	Cloud Logging (request + container)

Application modules use standard library / Flask logging patterns; there is no mandatory structured-log schema enforced repo-wide. `` if a future branch introduces OpenTelemetry exporters.

CI / quality signals

Signal	Source
Ruff / mypy	<code>ci.yml</code> lint job
Pytest results	LSP + backend jobs
Codecov (optional)	Python 3.13 matrix cell
AXIS coverage lcov	<code>axis-coverage-lcov</code> artifact
Playwright report	Uploaded on nightly/e2e failure
Security tests	<code>bun run test:security</code> (PWA job + nightly)

Usage metering (app-layer)

`backend/middleware/auth.py` tracks per-key:

- `calls_used` / `calls_limit` / tier
- `last_used` timestamps

These are **business metrics** for rate limits, not Prometheus time series — export them if you need dashboards.



Internals

Path	Role
backend/app.py	Health route, CORS, MAX_CONTENT_LENGTH
backend/middleware/auth.py	Key store, rate limit counters
backend/services/backtest.py	Backtest metrics objects for API responses
Dockerfile.api	HEALTHCHECK + gunicorn CMD
cloudbuild.yaml	Cloud Logging-only build logs
logs/ (repo)	Local combined/error log files if generated by tools — not the production contract

Invariants & edge cases

1. **Health ≠ readiness of evaluator warm caches.** A 200 on `/` does not mean the first `/run` will be fast (cold import / JIT / numba).
2. **Timeouts double-bind.** Gunicorn 60s and Cloud Run 60s — the tighter effective limit is still 60s.
3. **Scale-to-zero hides idle metrics.** No traffic ⇒ no samples; use synthetic uptime checks if SLOs matter.
4. **Coverage artifacts expire** (7–14 days retention on Actions uploads).
5. **Do not scrape secrets from logs.** API keys and admin tokens must never be logged at info level.

Worked examples

Local health loop

```
make run
curl -s localhost:8080/ | python -m json.tool
```

Docker health

```
docker inspect --format='{{json .State.Health}}' CONTAINER
```

Tail Cloud Run (gcloud)

```
gcloud run services logs read pynescrypt-pro-api --region us-central1 --limit 50
```



Failure modes

Symptom	Cause	Fix
Health green, <code>/run</code> 500	Exception in evaluator path	Inspect request logs; reproduce with payload
Worker kills	Soft timeout	Optimize script / raise timeout consistently
Missing CI artifact	Path wrong / tests passed	<code>if-no-files-found: ignore</code> hides absence
Rate limit false positives	Shared key / limit too low	Adjust tier limits; multi-key
Silent deploy failure	Build logging-only + unread	Check Cloud Build history UI

See also

- [GCP](#)
- [Docker](#)
- [Security](#)
- [API lifecycle](#)



SECURITY

API keys, CORS, body limits, secrets hygiene, Fernet metadata keys, and threat-aware defaults for PYNE ops.

SECURITY

Abstract

PYNE security is layered: **transport/hosting** (Cloud Run, containers), **application** (API keys, admin tokens, CORS, body size caps), **build secrets** (Fernet metadata keys, VSCE PAT), and **supply chain** (CI dependency installs, extension packaging). This page documents controls that exist in code and workflows — not a full threat model certification.

Conceptual model

```
flowchart TB
    NET[Internet] --> CR[Cloud Run allow-unauthenticated]
    CR --> CORS[flask-cors allowlist]
    CORS --> SIZE[MAX_CONTENT_LENGTH 5MB]
    SIZE --> AUTH[require_api_key]
    AUTH --> STORE[APIKeyStore hash + tiers]
    AUTH --> RUN[Evaluator /run]
    ADMIN[X-Admin-Token] --> KEYMGMT[key minting routes]
    BUILD[CI secrets] --> META[METADATA_KEY]
    BUILD --> VSCE[VSCE_PAT]
```

Interface surface

HTTP application controls (backend/app.py)

Control	Default / mechanism
Body size	<code>MAX_CONTENT_LENGTH = 5 * 1024 * 1024</code> (5 MiB) — DoS/OOM mitigation
CORS	<code>ALLOWED_ORIGINS</code> env (comma-separated); default includes product hosts + localhost regex
Methods	GET, POST only via flask-cors config
Headers	<code>Content-Type</code> , <code>Authorization</code> , <code>X-Admin-Token</code>
Credentials	<code>supports_credentials=False</code>

API keys (backend/middleware/auth.py)

- Keys stored as **hashes**, not raw secrets (store path via `API_KEY_STORE`)
- Tiers: `free` , `hobby` (5k), `pro` (50k), `team` (200k), `enterprise` (∞)
- Rate limit by monthly call counters (`calls_used` / `calls_limit`)
- Decorators: `require_api_key` , `require_admin_token`



Build / release secrets

Secret	Purpose
METADATA_KEY / _METADATA_KEY	Stable Fernet key for LSP metadata encrypt
VSCE_PAT	VS Code Marketplace / vsce package
GITHUB_TOKEN	Release asset upload
PYNESCRYPT_METADATA_KEY	Runtime decrypt env fallback

Container hardening (Dockerfile.api)

- Multi-stage build (no compiler toolchain in final image)
- Non-root appuser
- Production env flags
- Healthcheck without privileged ops

Internals

Path	Role
backend/app.py	CORS, body limit, blueprint registration
backend/middleware/auth.py	Key store, tiers, decorators
backend/middleware/key_store_redis.py / key_store_sqlite.py	Alternative stores
backend/middleware/schemas.py	Request validation schemas
src/pynescrypt/langserver/providers/metadata_decrypt.py	Encrypted metadata load
.github/workflows/*.yaml	Secret consumption
frontend security tests	bun run test:security in CI

Audit notes encoded in source

Comments in app.py reference a **2026-07-05 audit** (findings S1 body size, S7/S8 CORS). Prefer reading the code over assuming defaults never change.

Invariants & edge cases

1. Cloud Run --allow-unauthenticated does not mean open data. It means IAM does not gate invoke; the Flask app must.
2. Free tier with calls_limit=0 is special-cased as unlimited remaining in some helpers — verify tier semantics before production billing.
3. Localhost CORS regex permits any port on localhost/127.0.0.1 — fine for desk, wrong for prod ALLOWED_ORIGINS .
4. Fernet protects binary metadata packaging, not user script confidentiality.
5. Admin token routes are high privilege — rotate and store separately from user API keys.
6. Never log raw API keys. Hash lookups only.



Worked examples

Harden CORS for a single origin

```
python -m backend.app
```

Point key store at a volume

```
# ensure process user can write /data
```

Rotate metadata key (ops procedure sketch)

- 1. Generate new Fernet key; store as secret `METADATA_KEY`
- 2. Rebuild LSP with `CRYPTO_KEY` set
- 3. Ship new binary; retire old key after client upgrade window
- 4. Do not leave dual-key decrypt unless you implement it

Failure modes

Symptom	Cause	Fix
Browser CORS error	Origin not listed	Set <code>ALLOWED_ORIGINS</code>
413 Payload Too Large	Body > 5MB	Reduce OHLCV batch; raise limit knowingly
401/403 on <code>/run</code>	Missing/invalid key	Check <code>Authorization</code> header scheme
Rate limited	Tier exhausted	Upgrade tier / wait reset
Secret in fork PR logs	Misconfigured workflow echo	Never <code>echo</code> secrets; use masked env
Root-owned files in volume	Old image ran as root	Align UID with <code>appuser</code>

See also

- [Auth and keys \(API\)](#)
- [Metadata crypto](#)
- [Docker](#)
- [GCP](#)